

Article

A Phrase Fill-in-Blank Problem in a Client-Side Web Programming Assistant System

Huiyu Qi ^{1,*}, Zhikang Li ¹, Nobuo Funabiki ¹, Htoo Htoo Sandi Kyaw ¹ and Wen Chung Kao ²
¹ Department of Information and Communication Systems, Okayama University, Okayama 7008530, Japan; p8d81de9@s.okayama-u.ac.jp (Z.L.); funabiki@okayama-u.ac.jp (N.F.); htoohtoo@okayama-u.ac.jp (H.H.S.K.)

² Department of Electrical Engineering, National Taiwan Normal University, Taipei 10610, Taiwan; jungkao@ntun.edu.tw

* Correspondence: pbr17iqa@s.okayama-u.ac.jp

Abstract

Mastering *client-side Web programming* is essential for the development of responsive and interactive Web applications. To support novice students' self-study, in this paper, we propose a novel exercise format called the *phrase fill-in-blank problem (PFP)* in the *Web Programming Learning Assistant System (WPLAS)*. A PFP instance presents a source code with blanked phrases (a set of elements) and corresponding Web page screenshots. Then, it requests the user to fill in the blanks, and the answers are automatically evaluated through *string matching* with predefined correct answers. By increasing blanks, PFP can come close to writing a code from scratch. To facilitate scalable and context-aware question creation, we implemented the *PFP instance generation algorithm* in *Python* using *regular expressions*. This approach targets meaningful code segments in *HTML*, *CSS*, and *JavaScript* that reflect the interactive behavior of front-end development. For evaluations, we generated 10 PFP instances for basic Web programming topics and 5 instances for video games and assigned them to students at Okayama University, Japan, and the State Polytechnic of Malang, Indonesia. Their solution results show that most students could solve them correctly, indicating the effectiveness and accessibility of the generated instances. In addition, we investigated the ability of *generative AI*, specifically *ChatGPT*, to solve the PFP instances. The results show 86.7% accuracy for basic-topic PFP instances. Although it still cannot fully find answers, we must monitor progress carefully. In future work, we will enhance *PFP* in *WPLAS* to handle non-unique answers by improving answer validation for flexible recognition of equivalent responses.

Keywords: Web client programming; Web game; HTML; CSS; JavaScript; phrase fill-in-blank problem; regular expression; generative AI



Academic Editors: Galina Bogdanova, Malinka Spasova Ivanova and Todor Todorov

Received: 11 June 2025

Revised: 6 August 2025

Accepted: 12 August 2025

Published: 20 August 2025

Citation: Qi, H.; Li, Z.; Funabiki, N.; Sandi Kyaw, H.H.; Kao, W.C. A Phrase Fill-in-Blank Problem in a Client-Side Web Programming Assistant System. *Information* **2025**, *16*, 709. <https://doi.org/10.3390/info16080709>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Nowadays, with the proliferation of Internet technologies, *client-side Web programming* has become essential for building interactive and responsive browser-based user interfaces in Web applications with *HTML*, *CSS*, and *JavaScript* interactions [1]. However, for novice learners, understanding how these languages work together to produce meaningful “movement” and “dynamic behavior” on a Web page remains a significant challenge. Mastering the interplay between structure, styling, and event-driven logic is crucial for the development of effective, user-friendly Web applications.

Although computer science degree programs typically cover these technologies, a lot of students, especially those from non-computer science majors, lack sufficient exposure

to or hands-on experience with *client-side Web programming*. Furthermore, even in formal Web development courses, students may not truly master the skills without extensive hands-on practice with realistic and integrative tasks. As a result, there is a growing demand for scalable self-learning tools that support independent learning of practical *client-side Web programming* skills. These resources will help learners acquire basic front-end skills through independent practice, thereby bridging the gap from basic traditional programming languages (e.g., C or Java) to the dynamic environments of HTML, CSS, and JavaScript.

To address this need, we developed a Web-based *Web Programming Learning Assistant System* (WPLAS) as a self-learning platform that offers a sequence of programming exercises designed to progressively build student competencies. WPLAS provides several types of programming exercise problems that have different difficulty levels to meet different learning goals to master *client-side Web programming* through self-study. Existing exercise formats in WPLAS include the *Grammar-Concept Understanding Problem* (GUP), *Value Trace Problem* (VTP), *Code Modification Problem* (CMP), and *Element Fill-in-Blank Problem* (EFP). Any answer from a student is automatically marked in the system.

However, these exercises typically focus on isolated elements and do not require learners to construct complete functional code integrating multiple components. Therefore, learners may not reach the practical level of writing source codes from scratch.

In this paper, we introduce a new exercise format in WPLAS called the *Phrase Fill-in-blank Problem* (PFP) for *client-side Web programming*. A PFP instance presents a partial source code with multiple blanked-out phrases or sets of continuous elements, along with a screenshot of the corresponding Web page. The learner is expected to fill in the blanks to recreate the intended behavior, referring to remaining elements. This approach bridges the gap between elemental-level understanding and full code writing. Answer validation is still performed using *string matching* to ensure full automation and scalability.

To facilitate the widespread creation of these exercises, we propose a *PFP instance generation algorithm* that is implemented in Python 3.11.5 using *regular expressions* [2]. This algorithm applies rule-based techniques to extract meaningful code phrases from existing HTML, CSS, and JavaScript source codes. It can reduce the manual burden on instructors while enabling large-scale and reproducible exercise generation.

For evaluation purposes, we generated 10 PFP instances covering fundamental Web programming topics, as well as 5 additional instances derived from interactive browser-based video games. These game-based instances were selected because they involve complex interactions between contents, styles, and logics, making them an ideal choice for assessing comprehensive *client-side Web programming* understanding. These exercises were assigned to students from Okayama University, Japan, and Malang State Polytechnic, Indonesia, totaling over 60 participants. We validated the educational effectiveness of the proposed PFP by analyzing the correctness of solutions, the time required, and student feedback.

In addition, we investigated the problem-solving ability of *generative AI*, specifically ChatGPT [3], on the same 10 basic-topic PFP instances. It was found that ChatGPT achieved 86.7% accuracy. While many answers were correct, others were plausible alternative solutions or clearly incorrect. This comparison highlights both the current competence of AI models and the importance of designing exercise types like PFP that challenge learners beyond copy-paste behaviors.

In future works, we plan to refine the *PFP instance generation algorithm* to better match learner proficiency and improve answer validation mechanisms to handle equivalent responses more robustly. We also aim to explore an adaptive difficulty tuning method based on students' performance data to support personalized learning paths within WPLAS.

The rest of this paper is organized as follows: Section 2 discusses related works in the literature. Section 3 outlines core technologies and libraries adopted in this study. Section 4 presents the *phrase fill-in-blank problem (PFP)* for *client-side Web programming*. Section 5 presents the *PFP instance generation algorithm* for basic programming topics. Section 6 introduces the key phrase selection rules used in PFP instance generation, while Section 7 presents its extensions to game programming. Sections 8 and 9 evaluate the PFP instances through student application for basic programming topics and game programming. Section 10 investigates the PFP solution ability of *generative AI*. Section 11 concludes this paper with directions for future research.

2. Related Works in the Literature

In this section, we discuss related works in the literature.

2.1. Automated Programming Exercises

Here, we review works on automated programming exercises.

When programming exercises are created manually, educators often face challenges related to time efficiency, scalability, and the risk of inconsistency or human error. To address these issues, a range of automated systems has been developed to support the creation and evaluation of programming exercises [4].

Smith et al. also explored the use of automated fill-in-blank exercises in online learning environments [5]. Their findings showed that such formats can effectively evaluate learners' understanding of fundamental programming constructs. However, they also noted shortcomings in adaptability to varied student responses and the lack of personalized feedback, highlighting areas where current systems can be improved.

To broaden the accessibility and personalization of programming exercises, Jordan et al. explored the use of *ChatGPT* (GPT-3.5) to automatically generate non-English programming exercises [6]. Their work highlights the potential of AI-driven systems to support multilingual learners by tailoring exercise content to diverse linguistic contexts. This approach represents a significant advancement toward inclusive and globally adaptable programming education.

Expanding on the trend of AI-assisted learning, Denny et al. proposed the concept of *prompt problems*, a novel exercise format designed for the generative AI era [7]. Unlike traditional static tasks, *prompt problems* leverage AI-generated scaffolding to offer students flexible, adaptive, and interactive learning experiences. Their findings suggest that these exercises can dynamically adjust difficulty in response to student performance, promoting deeper engagement, critical thinking, and long-term learning progression.

Messer et al. conducted a systematic review of automated grading and feedback tools for programming education [8]. Their work categorizes and evaluates a wide range of systems, identifying emerging trends and limitations in scalability, feedback mechanisms, and integration with learning environments. Their findings provide a comprehensive overview of the current landscape and highlight key areas for further improvement.

Similarly, Daradoumis et al. analyzed student perceptions to refine the design of an automated assessment tool in online distributed programming [9]. Their results underscore the importance of aligning automation design with student expectations and learning needs, particularly in remote and blended learning contexts.

Duran et al. also explored the value of student self-evaluation in automated programming education [10], revealing how combining automated tools with metacognitive strategies can lead to more reflective and effective learning.

Papadakis [11] and Gulec et al. [12] have further contributed to this field by exploring gamified and game-based platforms that support automatic code evaluation and interactive

learning experiences. Their findings suggest that such systems can significantly increase student motivation and reinforce core programming skills in authentic contexts.

2.2. AI for Programming Education

Here, we review recent works on the application of *artificial intelligence (AI)* in programming education, with a focus on *large language models (LLMs)* such as *ChatGPT*.

Recent studies have evaluated the use of LLMs in classroom settings. Estévez-Ayres et al. examined LLM-based feedback generation in concurrent programming, reporting promising results, with some concerns about accuracy and alignment with learning goals [13].

Ma et al. presented a case study on student interactions with *ChatGPT* in Python courses, noting improved engagement but also recurring misconceptions [14].

Doughty et al. compared *GPT-4* and human-generated multiple-choice questions, finding comparable quality but emphasizing the need for careful validation [15].

Ghimire et al. [16] explored the use of *ChatGPT* in foundational courses, showing its usefulness in concept clarification and problem-solving, while also raising concerns about over-reliance.

Similarly, Silva et al. [17] highlighted benefits in debugging and learning support but cautioned against the erosion of critical thinking and academic integrity.

Popovici et al. [18] found that AI-generated explanations and feedback helped students in a functional programming course better understand key concepts.

Rahman et al. [19] further emphasized AI's role in enabling personalized learning through real-time support and content generation.

Lo et al. [20] reviewed *ChatGPT*'s influence on student engagement, noting its effectiveness in delivering context-aware feedback and enhancing self-paced learning.

Strielkowski et al. [21] discussed AI-driven adaptive learning as a scalable approach to personalization, identifying it as a key enabler for inclusive education.

Finally, Chang et al. [22] conducted a systematic review of AI-generated content tools like *ChatGPT*, *Copilot*, and *Codex*, outlining their advantages in coding assistance, debugging, and dynamic problem generation.

WPLAS aims to innovate in the use of the phrase fill-in-blank problem (PFP) in *client-side Web development*, especially for languages like *HTML*, *CSS*, and *JavaScript*. Unlike traditional systems that rely on static exercises, *WPLAS* offers dynamic support by providing hints for challenging problems and interactively introducing new syntax. One of the key innovations of *WPLAS* is its ability to present different types of PFPs suited to self-study and real-world development scenarios. These exercises are designed not only to test basic syntax and structure but also to simulate the complexity of actual Web development tasks, which often involve debugging, layout adjustments, and full code-writing challenges.

3. Review of Adopted Technologies

In this section, we briefly review the core technologies used in this study and highlight new contributions that these technologies make beyond our previous work.

3.1. Web Programming Learning Assistant System (WPLAS)

The *Web Programming Learning Assistant System (WPLAS)* was developed based on the *Programming Learning Assistant System (PLAS)* [23]. *PLAS* supports a wide range of programming languages (e.g., C, C++, and Java) through *GUP*, *VTP*, *BUP*, *CMP*, *EPF*, and *CWP* for self-learning. *WPLAS* continues this architecture but emphasizes *client-side Web programming* and introduces a new, more complex problem format.

3.1.1. Step-by-Step Study

The range of problem types in *WPLAS* will collectively advance students' understanding of *client-side Web programming* in a structured manner. Learners typically begin with *GUP* for grammar fundamentals, then progress through *VTP*, *BUP*, *CMP*, and *EFP* before encountering the newly introduced *PPF* and finally reaching *CWP*.

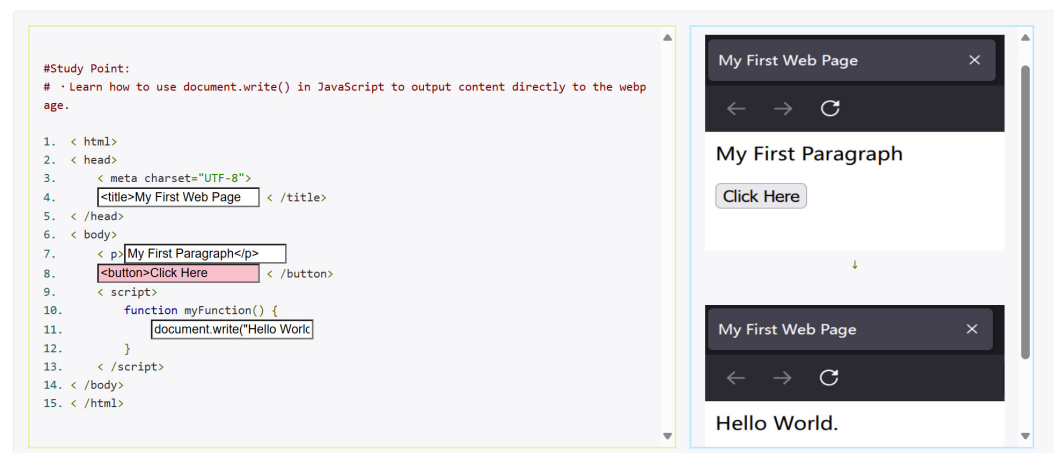
To reinforce dynamic programming concepts, we strategically place *PPF* as the second-to-last problem type, following *EFP*. Since *EFP* focuses on individual word blanks, it helps students become familiar with isolated syntax components and key elements in *client-side Web programming*. On the other hand, *PPF* requires learners to complete entire phrases, challenging them to understand the broader structural and behavioral aspects of the source code. This progression ensures that students first develop a granular understanding of syntax before tackling larger contextual gaps in source code.

When students reach the final stage of solving *CWP*, they are equipped with the necessary skills to write complete functional programs, integrating knowledge gained from all of the previous problem types. This structured learning pathway fosters a comprehensive grasp of *client-side Web programming*, enhancing both conceptual understanding and practical proficiency.

3.1.2. Answer Interface

Figure 1 shows a part of the answer interface. This interface runs on a Web browser and allows both online and offline use, since the answer marking is processed by running a *JavaScript* program on the browser. The correct answers in the interface are encrypted using *SHA256* to avoid cheating by students.

Problem #1



Answer

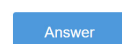


Figure 1. Interface of ID = 1 in PPF.

The left side of Figure 1 shows the source code. The right side shows eight questions generated from the source code that are related to the code behaviors and the basic *JavaScript* grammar.

After a student enters an answer in the blank form and clicks the "Answer" button shown in Figure 1, the answer-marking program checks the correctness of the submitted answer through *string matching*. If the answer is wrong, the background color of the blank turns *red*. Conversely, if the answer is correct, the background color changes to *white*.

Students have the flexibility of modifying and resubmitting answers as needed until they can reach correctness or giving up modifying them.

3.2. Regular Expressions for Answer Extraction

The implementation also utilizes *regular expressions* in *Python* to extract code patterns during the PFP generation process. Regular expressions assist in recognizing phrases suitable for masking and validating learner inputs against expected syntactic structures. The matching process using the *regular expression* library in a *Python* program is given as follows:

1. Import the *re* library.
2. Compile the *regular expression* using the *re.compile()* function and generate the *regular expression object* with the pattern string and optional flag parameters for use by the *match()* and *search()* functions.
3. Use the *search()* function to search the *HTML* file as a parameter one by one.
4. If every character of one element is matched with the pattern, the match is successful. Then, the matched element is added to the answer object using the *append()* function.

3.3. ChatGPT for Answer Uniqueness Verification

ChatGPT [3] is used as a supplementary tool to verify the uniqueness and appropriateness of answers for each PFP instance. By comparing outputs from our generator with those from *ChatGPT*, we identify whether multiple semantically valid answers exist. This verification process helps refine the algorithm to avoid ambiguity and improve the pedagogical value of each exercise.

4. Phrase Fill-in-Blank Problem

In this section, we introduce the *phrase fill-in-blank problem (PFP)* for *client-side Web programming*. It constitutes the main novel contribution of this study. The *PFP* is specifically designed to deepen students' understanding of *client-side Web programming* by requiring them to complete syntactically meaningful code phrases rather than individual keywords.

4.1. Overview of PFP

A *PFP* instance for *client-side Web programming* consists of a source code that integrates *HTML*, *CSS*, and *JavaScript* along with a series of questions. These blanks focus on the key phrases that describe both the static structure and the dynamic behaviors.

In a typical instance, dynamic responses involve interactions triggered by user actions (e.g., button clicks and form submissions), as well as real-time updates (e.g., textual content changes, style modifications, and element visibility toggles) applied to Web pages. These issues require learners to understand not only the syntax but also the underlying logic and behavior that drive such dynamic changes in *client-side Web applications*.

Unlike traditional fill-in-blank exercises targeting isolated elements, *PFP* tasks are based on basic and game-based functional source code with multiple blanked phrases. These phrases represent interactions that are essential in dynamic Web applications. The *PFP* format is designed to assess students' comprehension of the following:

- Code structure and logic flow across *HTML*, *CSS*, and *JavaScript*;
- Behavioral interactions between *UI* components and user input; and
- Syntax and semantics of common *client-side programming* patterns.

This type of the exercise problem is particularly suitable for evaluating conceptual understanding of interactive Web development and Web game logic, such as *DOM* manipulation, event handling, dynamic styling, and user feedback.

4.2. Candidates for Key Phrases

The fundamental aspect of generating a PFP instance lies on selecting appropriate phrases from the source code to serve as *key elements* for blanks. In this paper, we select the following key elements:

- HTML: id, text message (e.g., label messages);
- CSS: property name (e.g., color, display), selector (e.g., .class, #id); and
- JavaScript: reserved word (e.g., var, function), identifier (e.g., variable or function name), id, library class/method (e.g., Math.random()), text message used in DOM manipulation.

These key phrases are selected to represent meaningful components that contribute to the behavior of the code. Figure 1 illustrates an example of a PFP interface, where such phrases are masked as blanks and presented as structured questions in the right-hand panel.

4.3. PFP Instance Generation Procedure

The PFP instances used in this study are automatically generated from a source code using the PFP instance generation algorithm. Figure 2 illustrates the overall process of this generation method.

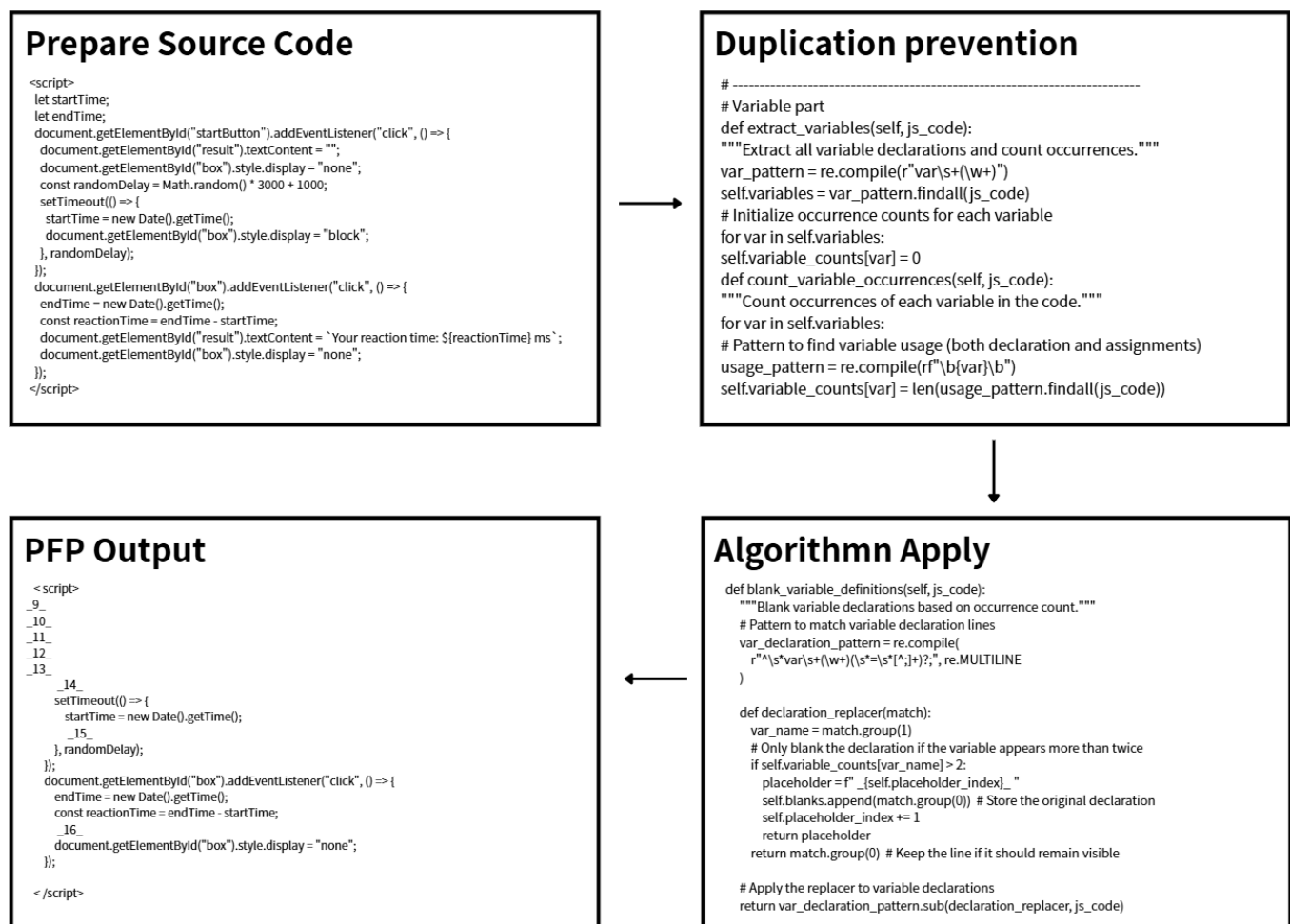


Figure 2. PFP generation procedure.

1. **Source code selection:** Select a *client-side Web programming* source code that includes interactive behaviors (e.g., event handling or DOM updates). This code can be sourced

from websites, textbooks, or educational materials. This forms the input for PFP instance generation.

2. **Duplication prevention:** To avoid confusions caused by repeating patterns, the algorithm first detects variable and function declarations and counts their occurrences. Only those that appear multiple times are considered for blank generations, ensuring that the resulting blanks are meaningful and not trivially obvious.
3. **Algorithm application:** Using the filtered variable information, the algorithm applies the *regular expression*-based rules to extract key phrases. These phrases, such as variable declarations, expressions, or event handlers, are replaced with placeholders. The original phrases are stored as correct answers.
4. **PFP instance output:** The modified source code with placeholders is output as a PFP instance. This version is ready for integration into the answer interface and serves as the core of the interactive learning activity.

This automatic PFP instance generation approach reduces the manual burden of educators and enables scalable content creation. To ensure the validity and uniqueness of each generated problem, we employ basic-topic PFP instances and compare the expected answers with those generated by *ChatGPT*. This manual verification step helps confirm that each blank has a well-defined and unique correct answer, thereby enhancing the educational value and reliability of the exercises.

5. PFP Instance Generation Algorithm

In this section, we present the *PFP instance generation algorithm* using key phrase selection rules.

5.1. Algorithm Overview

The PFP instance generation algorithm follows a consistent five-step procedure, as outlined in Figure 3. This structured approach ensures both extensibility and consistency across various key phrase types, such as *JavaScript* methods, *HTML* tags, and *CSS* properties.

1. **Pattern Definition:** The algorithm begins by defining a detection pattern tailored to each key phrase type. *Regular expressions* are used to identify syntactic units that represent meaningful educational targets, such as *JavaScript* reserved words, *HTML* tags, or *CSS* properties. These patterns are crafted to capture relevant code segments that reflect the core concepts of *client-side Web development*.
2. **Pattern Matching:** The source code is then parsed line by line, and the algorithm searches for matches using the defined patterns. The detection is context-sensitive, for example, distinguishing a reserved word within a loop from one inside a function body.
3. **Phrase Replacement and Storage:** Once a matching is found, the corresponding code fragment is replaced with a placeholder (i.e., a blank) in the instance output. Simultaneously, the original phrase is stored in a replacement list to be presented as the answer set.
4. **Duplication Handling:** To maintain pedagogical efficiency, the algorithm tracks previously replaced phrases (e.g., identical id values or repeated function names) and avoids creating redundant blanks. This reduces cognitive overload and improves exercise variety.
5. **Instance Output:** Finally, the modified source code, now containing blanks, is output as a new PFP instance. Each instance is designed to retain the integrity of the original program logic while encouraging students to infer the missing phrase based on context.

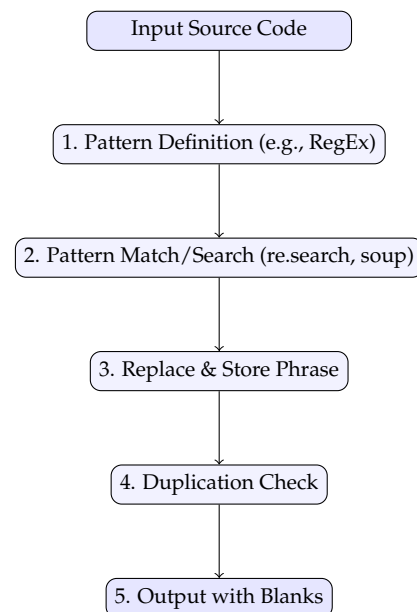


Figure 3. Detection workflow for generation of phrase blanks from client-side source code.

This workflow is uniformly applied across all the phrase types described in the following subsections. Each of them illustrates how the five-step detection method is adapted to a specific language construct.

5.2. Redundant Blank Prevention

When generating phrase blanks automatically, we encountered specific challenges related to variable occurrences and redundancy in the generated blanks. One notable issue arises when multiple instances of the same variable name appear in the source code. Since these variables are invoked multiple times throughout the program, blanking out all occurrences simultaneously would significantly increase the difficulty level, making it harder for students to deduce the correct answers.

To address this issue, our algorithm first counts the number of occurrences of each variable in the game program. Instead of blanking all instances, we implement a selective blanking strategy based on the frequency of each variable:

- If a variable appears *twice*, we blank only one occurrence to provide sufficient context while still assessing comprehension.
- If a variable appears *three times*, we selectively blank the *first and third occurrences*, maintaining the challenge of recognizing recurring variables without making the exercise overly difficult.
- If a variable appears *four or more times*, we blank the *first, third, and last occurrences* to ensure students engage with different parts of the code while still preserving some references for contextual understanding.

This adaptive approach ensures that PFP instances remain both *challenging* and *solvable*, preventing excessive blanking from making questions too ambiguous while still reinforcing students' ability to track variable usage across different sections of the program.

5.3. Details of Pattern Definition and Matching

The PFP instance generation algorithm relies on rule-based pattern matching to identify relevant code segments in *HTML*, *CSS*, and *JavaScript* for blanks. Therefore, we describe the details of **1. Pattern Definition** and **2. Pattern Matching** in the algorithm overview.

Our algorithm employs a structured approach to identify key phrases in the source code, forming the central component of the algorithm. It applies the following eight

fundamental selection rules, systematically pinpointing critical programming constructs. Each selected phrase becomes a “blank” in the final *PPF* instance, ensuring that the generated exercise emphasizes essential *HTML*, *CSS*, and *JavaScript* concepts for integrated *client-side Web development*.

1. **JavaScript Library Class/Method Element:** Our *PPF* instance generation algorithm includes a specific rule to detect method usage. As illustrated in Figure A2, it uses a regular expression to identify calls to `document.getElementById("ID")`, which is a typical method for accessing *HTML* elements via the `id` attribute. When this pattern is matched, the corresponding code fragment is replaced with a blank, prompting learners to recall how to retrieve *DOM* elements in *JavaScript*. To prevent redundant blanks for the same element, the algorithm keeps track of already processed *IDs* and skips any repeated occurrences. This avoids unnecessary repetition and keeps the exercise streamlined and pedagogically effective.
2. **JavaScript Reserved Word Element:** *JavaScript* includes a set of reserved words (e.g., `if`, `for`, `return`, and `function`) that cannot be used as identifiers or variable names. These words form the backbone of the language’s grammar, signifying specific operations or structures, such as loops, conditionals, and functions. Selecting such reserved words as blank phrases prompts learners to recognize fundamental *JavaScript* syntax and control-flow structures. The implementation for detecting words like `alert` is shown in Appendix A.1 (Figure A1).
3. **JavaScript Identifier Element:** Identifiers in *JavaScript*, such as variable names, function names, or object properties, are crucial for maintaining readable and modular code. When selecting identifiers (e.g., `score`, `updateGame()`, and `playerName`), the generated blanks require students to track and interpret variable states or function calls. This underscores the importance of naming conventions, scope understanding, and the flow of data throughout a script. The detection approach for variable declarations using `let` or `var` is provided in Appendix A.3 (Figure A3).
4. **Id Name Element:** *HTML* `id` attributes uniquely identify *DOM* elements for *JavaScript* manipulation (e.g., `<div id="grid">`). Selecting `id` names as blanks forces students to connect the *layout structure* (*HTML*) with the *interactive logic* (*JavaScript*), demonstrating how client-side code references specific elements for reading or updating of content. Mastering `id` usage is vital for dynamic Web pages, especially in games where event handling and score tracking often hinge on precise *DOM* operations. The detection logic for tags such as `<div id="grid">` is illustrated in Appendix A.4 (Figure A4).
5. **Text Message Element:** In both *HTML* and *JavaScript*, text is often used to show messages. For example, `scoreDisplay.textContent = score` conveys dynamic information to users. This kind of message can be written directly inside an *HTML* tag, such as `p` or `h1`, or can be updated using *JavaScript* code such as `scoreDisplay.textContent = score`. The approach to extracting messages within tags like `<h1>`, `<h2>`, and `<button>` is shown in Appendix A.5 (Figure A5).
In *PPF* instances, these text parts are sometimes made blank so that students need to think about what message should be there. This helps students understand how important a text message is for giving users feedback, like showing scores or game status. In games, these messages are very important to keep the user engaged.
6. **CSS Syntax Element:** *CSS* rules (e.g., `background-color`, `display`, and `margin`) define a Web page’s visual presentation. Highlighting *CSS* syntax as blanks helps learners recall styling fundamentals and how small changes to properties can significantly alter game interfaces. In an interactive *client-side Web game*, aesthetics often guide user flow and clarity, making it crucial for students to master correct *CSS* syntax. The code for detecting relevant *CSS* properties appears in Appendix A.6 (Figure A6).

7. **HTML Tag Element:** *HTML* tags (e.g., `<title>` and `<canvas>`) provide the structural foundation of a Web page or game interface. Selecting them as blanks, the *PFP* generation tests students on the correct usage and context of these tags. It also reveals how different tags can be crucial for specific functionalities (like `<canvas>` for drawing). The detection logic, including regular expressions to find specific tags, is provided in Appendix A.7 (Figure A7).
8. **HTML Attribute Element:** *HTML* attributes (e.g., `class`, `src`, and `autoplay`) add details and functionalities to tags. In *client-side Web games*, attributes might control media playback, link CSS classes for styling, or set default behaviors (like `autofocus` on input fields). By removing these attributes and asking students to fill them in, the exercise underscores how attributes fine-tune the element's behavior and styling beyond the basic tag itself. The code for detecting attributes like `onClick` is shown in Appendix A.8 (Figure A8).

6. Key Phrase Rules for Game-Based PFP Generation

In this section, we describe the extended set of *key phrase selection rules* used in the *PFP instance generation algorithm* to support interactive Web game applications.

6.1. Rule Definitions

To support interactive Web game applications, we extend the *key phrase selection rules* used in the *PFP instance generation algorithm*. These rules are designed to target typical constructs found in *client-side game programming*, as studying game source code plays an important role in learning interactive Web programming.

1. **User Input Handling:** Games depend heavily on user interactions like mouse clicks or keyboard input. This rule captures event listeners (e.g., `addEventListener("click", ...)`), helping learners connect user actions to functional responses in the game.
2. **Event Handling Function:** Many games define specific callback functions such as `startGame()` or `handleCollision()`. This rule selects such handlers, challenging learners to understand event-driven programming.
3. **Loop Statement Loops** such as `for` or `while` are frequently used in games to animate objects or check repeated conditions. This rule detects loop constructs and replaces them with blanks to train students in constructing correct and efficient repetition structures.
4. **Dynamic Value Calculation Expression:** Game codes often include dynamic values such as scores, timers, or health bars that are continuously updated. This rule identifies expressions involving operations like `score++` or `lives--`. By turning them into blanks, learners are asked to recall the logic required to reflect game state changes.
5. **Switch-case Construct:** To handle multi-branch logic (e.g., selecting difficulty modes or game states), `switch-case` structures are used. The corresponding rule targets these constructs, promoting comprehension of conditional branching beyond `if-else`.
6. **Dynamic Animation and Styling:** Lastly, visual feedback is key in game UX. This rule targets direct style changes or animations (e.g., `element.style.left = ...`), encouraging learners to understand how code controls appearance and motion.

6.2. Rule Application Example

As illustrated in Figure 4, a single game script can simultaneously include multiple types of key phrases. In the *Whack-a-Mole* example, for instance, the code snippet features a dynamic value update (`score++`), user input handling via an event listener (`addEventListener("click", ...)`), an event-handling function (`getElementById("startB")`), and a loop mechanism (`setInterval(...)`). This example highlights how these constructs

are often interwoven in real-world interactive game programming, emphasizing the need for comprehensive rule coverage in PFP instance generation.

Problem #5

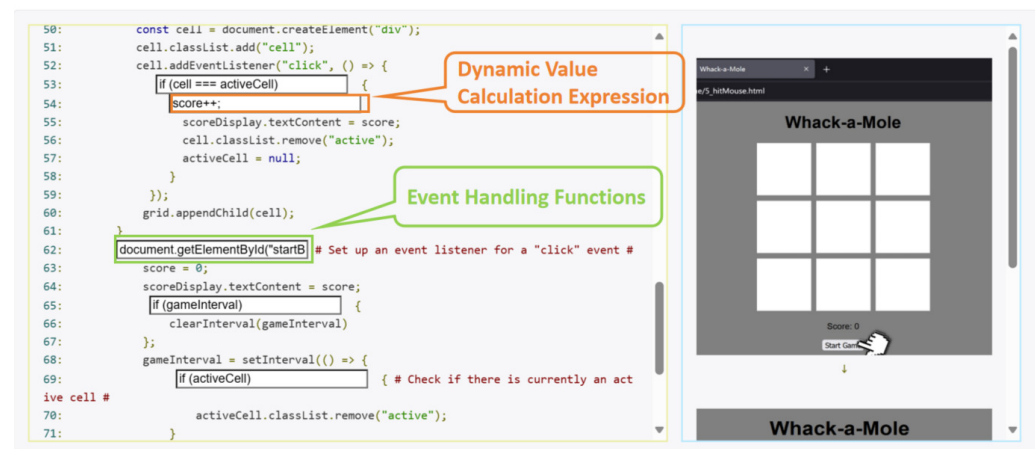


Figure 4. Example of game-based PFP with multiple rule types.

The six game-specific rules ensure that the generated *PFP* instance *pinpoints* the constructs essential in *client-side Web game programming*. By actively recalling and applying each piece of logic or styling, students can gain deeper insights into real-time user interactions, game-specific management, and interface feedback.

7. Algorithm Extensions to Game Programming

In this section, we extend the *PFP instance generation algorithm* to handle source-code examples from video games developed using *client-side Web programming* based on the game-specific rules defined in Section 6. *Game programming* plays a valuable role in programming education, as it demands tight coordination among inputs, control logic, and visual outputs. We selected game-based scenarios because they naturally incorporate *HTML*, *CSS*, and *JavaScript*, providing learners with a comprehensive and practical environment to apply and deepen their Web programming skills.

7.1. Reference Guide

For each *PFP* instance of *game programming*, we prepare a reference guide to help students solve it, since the instance can often be challenging due to the intricate interactions between *HTML*, *CSS*, and *JavaScript*. This guide provides guidance on key game mechanics, coding patterns, and troubleshooting strategies. It is accessible through the menu interface, as shown in Figure 5.

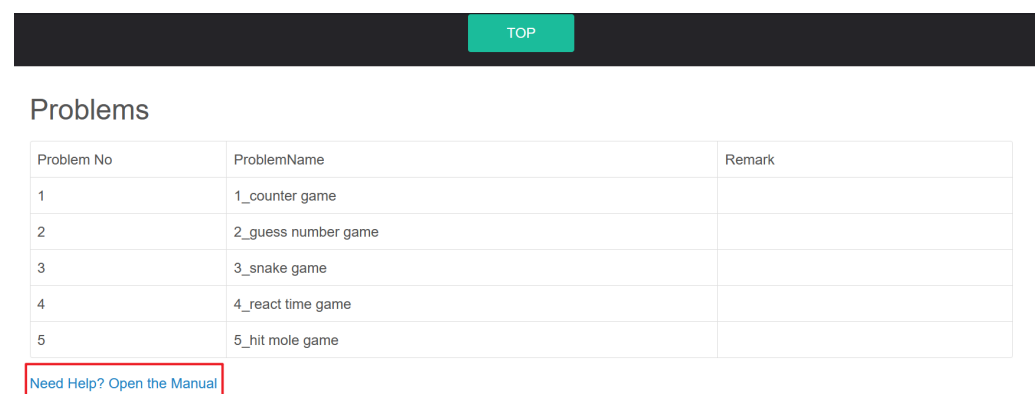


Figure 5. Menu interface.

Figure 6 displays the main interface of the reference guide. It provides an overview of the system and introduces key concepts related to this instance. It offers essential hints on the syntax used in this game-based *PPF* instance, helping students understand the structure of *HTML*, *CSS*, and *JavaScript* within the provided exercises.



Figure 6. Interface of reference guide.

Figure 7 showcases the section in the reference guide dedicated to *CSS*. Since styling plays a crucial role in *client-side Web game programming*, it provides a reference on common *CSS* attributes, selectors, and formatting rules. It explains different ways to define *CSS*, including inline, internal, and external styles. It also highlights key properties like background color, text alignment, and cursor properties, helping students understand how styling elements affect the appearance and behavior of game components.

• CSS

CSS Attributes: Follow the key-value format.

Three Ways to Define CSS: Inline, Internal, External.

However, in these five games, we only use **Internal CSS** to style the elements.

This means that all the styles are written inside the `<style>` tag within the `<head>` section of the *HTML* document.

CSS Selectors: There are three main ways to select elements in *CSS*.

- `#id` - Selects an element by its unique ID.
- `.class` - Selects elements that share the same class name.
- `element` - Selects all elements of the specified type.

Example:

```
<style>
/* Selects the entire body element */
body {
  background-color: lightgray;
}

/* Selects all elements with class 'div' */
.div {
  color: blue;
}

/* Selects all
elements */
p {
  font-size: 18px;
}
</style>
```

Here are some example attributes:

Examples:

- `background-color` - Defines the background color of an element. It accepts:
 - **Predefined Color Names** (e.g., red, blue, green)
 - **HEX Values** (e.g., #FF0000 for red, #00FF00 for green, #0000FF for blue)
 - **RGB Values** (e.g., rgb(255, 0, 0) for red, rgb(0, 255, 0) for green, rgb(0, 0, 255) for blue)
 - **HSL Values** (e.g., hsl(0, 100%, 50%) for red, hsl(120, 100%, 50%) for green, hsl(240, 100%, 50%) for blue)
- `cursor` - Controls the appearance of the mouse pointer when hovering over an element. Options include:
 - `pointer` - Indicates a clickable item, commonly used for buttons and links.
 - `default` - Resets the cursor to the system's default arrow.
 - `crosshair` - Displays a crosshair, often used in design and drawing tools.
 - `move` - Indicates that an element can be dragged.
 - `text` - Changes the cursor to a text selection icon, commonly used for editable text areas.
 - `wait` - Displays a loading spinner, indicating that the user should wait.
 - `help` - Shows a question mark, typically indicating help information is available.
 - `not-allowed` - Displays a forbidden sign, indicating an action is not possible.
- `font-family` - Specifies the typeface used for text rendering. It supports multiple values as fallbacks in case the preferred font is unavailable.
 - `font-family: Arial, Helvetica, sans-serif;` - Uses Arial if available, otherwise Helvetica, and defaults to any sans-serif font.
 - `font-family: 'Times New Roman', Times, serif;` - Uses Times New Roman if available, otherwise Times, and defaults to any serif font.
 - `font-family: 'Courier New', Courier, monospace;` - Uses Courier New if available, otherwise Courier, and defaults to a monospace font.
- `text-align: center / left / right;` - Controls the horizontal alignment of text within its container. `center` aligns the text in the middle, `left` aligns it to the left, and `right` aligns it to the right.

Figure 7. CSS reference in reference guide.

7.2. Source Code

To illustrate how a generated PFP instance appears to students, we provide an example of an *output text file* produced by the proposed algorithm. Instead of presenting the full source code within the main body, we show a representative screenshot to emphasize the learning interface and interaction in Figure 8. The corresponding output file, with the numbered blanks that highlight the detected key phrases, is presented in Appendix B. The interface presents these blanks in context, allowing learners to read the surrounding code and infer the correct phrase. Each blank is accompanied by a hint. Upon submitting answers, students receive immediate correctness feedback. This approach fosters active learning and supports self-assessment in *client-side Web programming*.

Problem #5

```

#Study Point:
# Understanding the core mechanics of a Whack-a-Mole game, such as random mole appearance
s, user interaction, and score calculation.
# Managing game state using variables and event-driven programming.

1:< !DOCTYPE html>
2:< html lang="en">
3:< head>
4:  < meta charset="UTF-8">
5:  < meta name="viewport" content="width=device-width, initial-scale=1.0">
6:  
7:  < style>
8:    body {
9:      font-family: Arial, sans-serif;
10:      # Specify the alignment of the text #
11:      # Set the background color #
12:    }
13:    #grid {
14:      display: grid;
15:      grid-template-columns: repeat(3, 100px);
16:      gap: 10px;
17:      justify-content: center;

```

Show Video

Answer

Answer

Figure 8. PFP instance interface with multiple blanks and contextual hints.

7.3. Blank Analysis

Table 1 lists 10 of the 20 blanks in this example instance in Appendix B (Figure A9), alongside the corresponding original code snippets and the rationale for their selection based on the key phrase rules. Each blank is tied to a unique question, prompting the learner to fill in the missing piece. Each question covers an essential concept, such as text alignment, a styling property, dynamic interaction event, or variable declaration, that is critical for the *Whack-a-Mole* game in *client-side Web programming*.

Table 1. Blank analysis in “Whack-a-Mole” instance (part).

Blank ID	Excerpt or Code Fragment	Rule Type	Line #	Remark
1	<code>_1_ (<title>Whack-a-Mole</title>)</code>	HTML Tag	5	Document title setup
2	<code>_2_ (text-align: center;)</code>	CSS Syntax	12	Centers body text
3	<code>_9_ (Whack-a-Mole</h1>)</code>	HTML Element/Text	37	Main heading text
4	<code>_10_ (let activeCell;)</code>	JavaScript Identifier	44	Tracks the current active cell
5	<code>_13_ (const scoreDisplay = document.getElementById("score");)</code>	JavaScript DOM Reference	48	Points to the score display
6	<code>_14_ (for (let i = 0; i < 9; i++) { ... })</code>	Loop Statement	50	Generates 9 game cells
7	<code>_15_ (if (cell === activeCell))</code>	Condition (Game Logic)	53	Checks if user clicked the mole
8	<code>_16_ (score++;)</code>	Dynamic Value Calculation	54	Increments the player score
9	<code>_17_ (document.getElementById("startButton")...)</code>	Event Handling	60	Sets up the start-button click

7.4. Role of Each Blank

Each blank in this PFP instance makes students recall or apply essential concepts in *client-side Web game programming*. To fill in these placeholders, students must connect knowledge of *HTML structure*, *CSS rules*, and *JavaScript logic* in a single, coherent exercise.

- **Styling blanks (e.g., `_2_`, `_3_`):** These blanks force students to recall crucial CSS properties, such as `text-align`, `background-color`, or `border`, and understand how they contribute to the overall look of the game.
- **DOM manipulation blanks (e.g., `_13_`):** Through referencing commands like “`document.getElementById(“grid”)`” or “`document.createElement(“div”)`”, these blanks highlight how *JavaScript* dynamically retrieves and modifies *HTML* elements.
- **Event-handling blanks (e.g., `_17_`):** Questions focused on click listeners (e.g., `document.getElementById(“startButton”).addEventListener(“click”, ...)`) require students to recall how user actions in a browser trigger specific callback functions. This is at the heart of real-time interactions in *client-side Web game programming*.
- **Dynamic game-state management blanks (e.g., `_15_`, `_16_`):** These placeholders revolve around loop statements, conditional checks (e.g., ‘if (cell === activeCell)’), and score increments. Prompting learners to fill in logic like ‘score++’ or random index calculations, the PFP instance fosters a practical understanding of how real-time game updates happen in a browser environment—reinforcing the interplay between user interactions, DOM updates, and styled visuals.

In the game-based instances, each blank is presented as an input field accompanied by a contextual hint. Upon submission, the system instantly provides correctness feedback, encouraging learners to explore, revise, and reinforce their understanding.

8. Evaluation of Basic PFP Topics in Web Programming

In this section, we evaluate the proposal for basic *client-side Web programming* topics. We generated 10 PFP instances by applying the *PFP instance generation algorithm* to source codes collected from the website and assigned them to 20 students from Okayama University, Japan, who had not studied *client-side Web programming* formally.

8.1. Generated PFP Instances

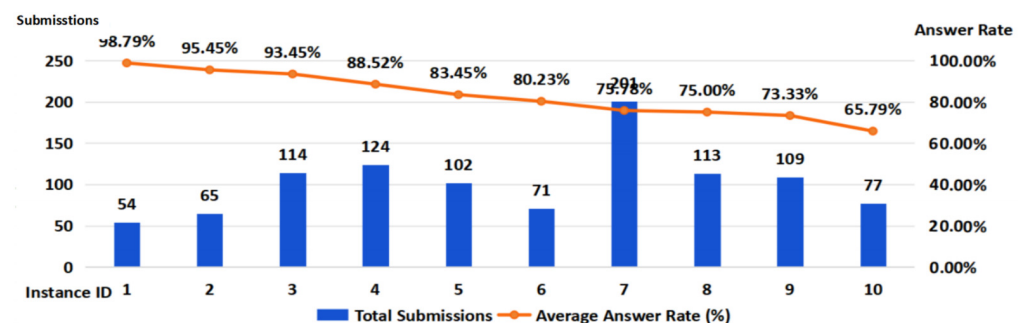
Table 2 shows the topic, the total number of lines, the total number of *JavaScript (JS)* lines in the source code, and the total number of blanks for each of the 10 generated PFP instances.

Table 2. PFP instances for basic topics.

ID	Topic	Total # of Lines	JS Lines	Blanks
1	Button Function	15	5	5
2	Easy Multiplication	15	6	4
3	Showing Date	15	5	5
4	Character Counter App	21	6	5
5	Selected Values	23	10	8
6	Object	25	11	8
7	Warning Message	26	6	7
8	Color Selector	27	11	7
9	Password Check App	32	13	10
10	Custom Timer	49	20	14
Total (Average)		248 (24.8)	93 (9.3)	73 (7.3)

8.2. Solution Results of Individual Instances

First, we analyze the solution results of the 10 PFP instances individually. Figure 9 shows the average correct answer rate and the total number of answer submissions by the 20 students for each instance. The average correct answer rate for the 10 instances is 82.98%, indicating that the majority of the answers submitted by the students were correct. This success rate reflects the effectiveness of the exercise design. Additionally, the average number of submissions per instance is 103, demonstrating consistent engagement across all instances. The instances at ID = 1 resulted in the best rate of 100%.

**Figure 9.** Solution results of individual instances for basic topics.

However, despite the higher engagement, the correct answer rate for instance ID = 7 was slightly lower, at 75.78%. A further analysis will be conducted to identify potential improvements.

The ID = 7 instance involves creating a dynamic character counter where the number of characters typed in a `<textarea>` is displayed in real time using *JavaScript*. To accomplish this, students must understand event-driven programming, specifically the on-input event, and how to retrieve and manipulate *DOM* elements using methods like `document.getElementById`. This requires familiarity with *JavaScript* syntax and the interaction between *HTML* elements and *JavaScript* code, which can be challenging for beginners.

8.3. Solution Results of Individual Students

Next, we analyze the solution results of the 20 students individually. Figure 10 illustrates the average correct answer rate and the total number of answer submissions across the 10 instances for each student. The graph is organized in descending order of student performance. The overall average correct answer rate is 85.75%, with an average of 51.50 submissions per student. One student demonstrated exceptional performance, achieving a perfect 100% accuracy rate. This student completed all 10 instances with only

12 submissions, indicating just two mistakes throughout the exercise. In contrast, another student achieved a significantly lower accuracy rate of 65% despite submitting answers 178 times. This indicates frequent errors and possibly a lack of understanding or preparation in programming concepts.

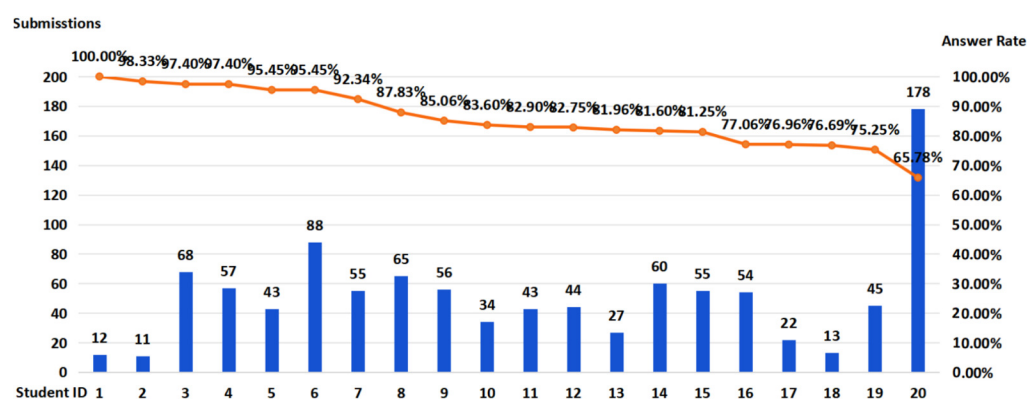


Figure 10. Solution results of individual students for basic topics.

The results indicate that the *PFP* instances for basic topics are accessible, even for students with minimal or no formal training in *client-side Web programming*. This suggests that the proposed algorithm successfully creates *PFP* instances that are effective for reinforcing basic programming concepts without overwhelming the learners.

9. Evaluation of Game PFP Topics in Web Programming

In this section, we evaluate the proposal for game programming. We generate five *PFP* instances by applying the extended *PFP instance generation algorithm* to source codes downloaded from websites and assign them to students from Okayama University, Japan, and State Polytechnic of Malang, Indonesia, who have not studied *client-side Web programming* formally.

9.1. Generated PFP Instances

Table 3 shows the topic, the total number of code lines, the total number of *JavaScript* (*JS*) lines in the source code, and the total number of blanks for each instance.

Table 3. *PFP* instances for game programming.

ID	Topic	Total # of Lines	JS Lines	Blanks
1	Counter Game	46	7	11
2	Guess Number Game	66	17	21
3	Snake Game	103	58	17
4	React Time Game	61	19	16
5	Hit Mole Game	80	36	20
Total (Average)		356 (71.2)	137 (27.4)	85 (17)

9.2. Solution Results of Individual Instances

Figure 11 illustrates solution results of the five video game *PFP* instances individually. The bar graph represents the total number of submission attempts per instance, while the line graph shows the corresponding average correct answer rates.

Instance ID = 1 achieved the highest accuracy, with a correct answer rate of 95.04%, indicating that it was relatively easy for most students. Instance ID = 2 recorded the highest number of submissions, with a total of 105 and slightly lower accuracy of 77.06%, suggesting moderate difficulty but strong engagement. Instances ID = 3 and ID = 4 exhibited a gradual

decline in both submission count and accuracy, with correct answer rates of 76.50% and 70.17%, respectively, suggesting increasing difficulty or a decrease in student motivation.

Notably, instance ID = 5 stands out, with a sharp drop in accuracy to 15.51%, despite having a similar number of submissions, totaling 36, compared to earlier instances. This suggests that instance ID = 5 posed a significant difficulty for learners, potentially due to complex logic or unfamiliar concepts in the source code. Additionally, the test duration was limited to approximately one hour, and as instance ID = 5 was positioned later in the sequence, many students may have run out of time, which could have further contributed to the lower accuracy rate.

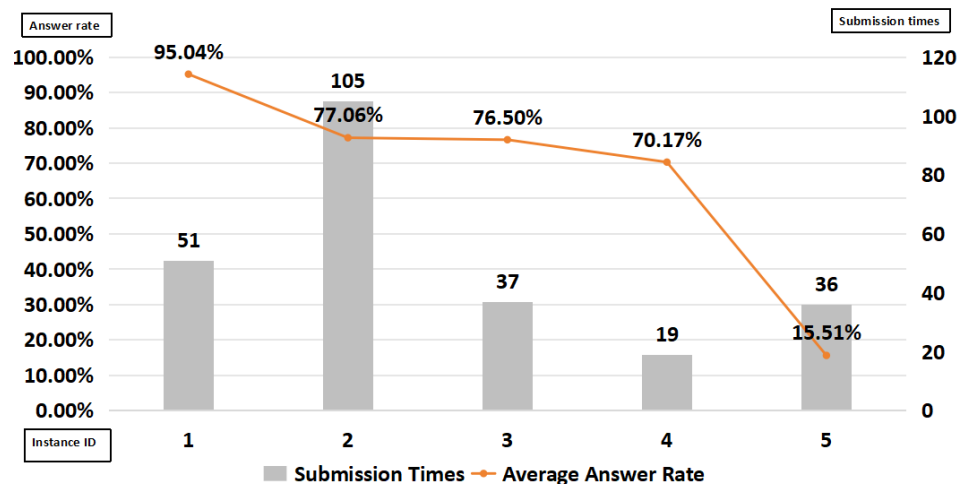


Figure 11. Solution results of individual instances for game programming.

9.3. Solution Results of Individual Students

Figure 12 shows the solution results for individual students, comparing their average correct answer rates with their total number of submission attempts. The graph provides insights into both student performance and engagement throughout the game-based PFP instances.

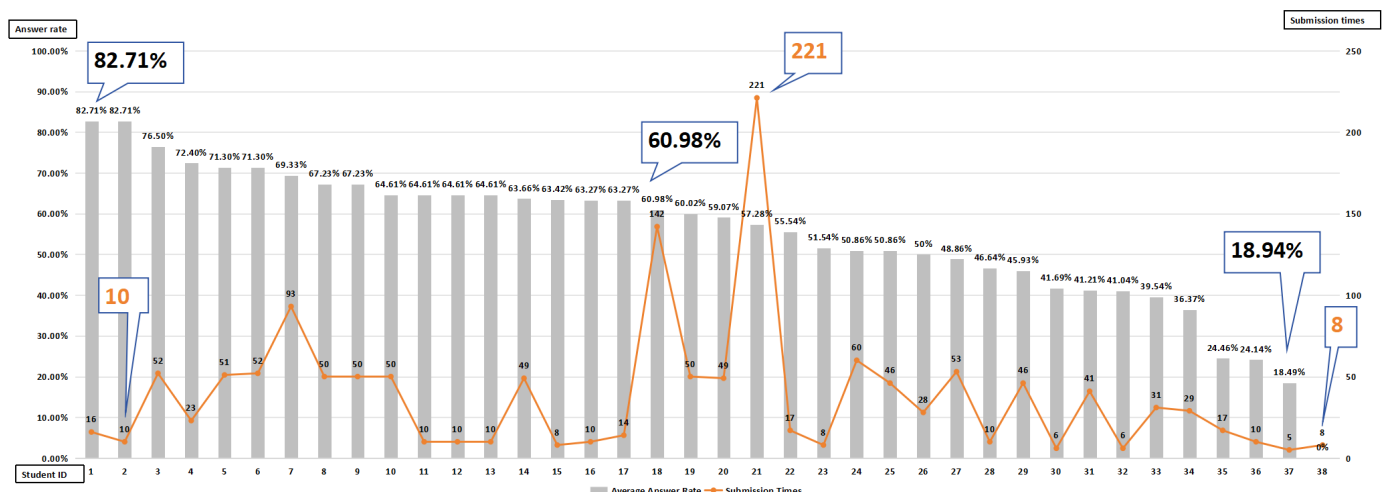


Figure 12. Solution results of individual students for game programming.

Students ID = 1 and ID = 2 achieved the highest accuracy rates of 82.71%, suggesting strong proficiency in *client-side Web programming* concepts. Student ID = 18 also demonstrated a relatively high performance of 60.98%, with the highest submission count of 221, indicating a high level of engagement and persistence. On the other hand, students like

ID = 37 and ID = 38 recorded significantly lower accuracy rates of 18.94% and 0%, respectively. This may suggest that these students encountered difficulties in understanding the content or ran out of time during the test.

It is worth noting that some students submitted relatively few answers but still achieved high accuracy, while others submitted more but had lower correctness.

10. Solution Ability of Generative AI

In this section, we investigate the ability of *generative AI* to solve the PFP instances. Recent advances in *generative AI*, particularly *large language models (LLMs)* such as *ChatGPT*, have significantly transformed the landscape of programming education. Students now have unprecedented access to tools that can generate solutions instantly, raising both opportunities for enhanced learning and concerns over academic integrity.

10.1. Solution Results for Basic Programming Topics

First, we prompted *ChatGPT* with the 10 basic topic instances with the related screenshots. Here, we explicitly asked two questions: “What are the answers for these blanks?” and “Are there any alternative answers for each blank?”.

As the response, Table 4 shows that *ChatGPT* correctly filled a total of 65 blanks among 75 in the 10 PFP instances, achieving an overall precision of 86.67%. This result suggests that *ChatGPT* exhibits strong performance in handling basic programming concepts such as button interaction, simple arithmetic, date formatting, and value selection.

Table 4. *ChatGPT* accuracy on basic topics.

ID	Topic	Blanks	Correct Blanks	Incorrect Blanks	Accuracy
1	Button Function	5	5	0	100%
2	Easy Multiplication	4	2	2	50%
3	Showing Date	5	5	0	100%
4	Color Selector	7	4	3	57.14%
5	Selected Values	8	8	0	100%
6	Object	8	5	3	62.5%
7	Character Counter Application	7	7	0	100%
8	Warning Message	7	7	0	100%
9	Password Check Application	10	9	1	90%
10	Custom Timer	14	13	1	92.86%
Total (Average)		75 (7.5)	65 (6.5)	10 (1)	(86.67%)

The PFP instances for basic topics are simpler in structure than those for game-based instances. As shown in Table 4, *ChatGPT* achieved 100% accuracy in 5 out of the 10 instances, indicating that it performs exceptionally well when the correct answer is unique and aligned with common coding convention. In this case, *ChatGPT* can easily infer the correct answer using the source code and accompanying screenshots.

Some answers differ from the ones in the original source code. It is found that many of them share common features or patterns. The differences are not intended to be arbitrary. Rather, they often reflect reasonable alternatives that are consistent with widely used programming practices.

Table 4 shows that the “Easy Multiplication” instance had the lowest accuracy rate, at 50%. Specifically, the original correct answers for the third and fourth blanks are `return a * b;` and `document.getElementById(“demo”).innerHTML = myFunction(4,3 );`. However, *ChatGPT* generates `document.getElementById(“demo”).textContent = a * b;` and `myFunction(3,4);`.

While these answers are syntactically valid and correct, they deviate from the expected solutions. The use of **textContent** instead of **innerHTML** and the reversal of **function parameters** suggest that *ChatGPT* prioritizes general coding familiarity over exact task-specific matching, which is critical in educational environments.

10.2. Solution Results for Game Programming

Next, we prompted *ChatGPT* with the five game programming instances with the related screenshots. Table 5 shows that *ChatGPT* correctly filled 68 out of 85 blanks across five PFP instances, confirming its strong baseline competence in *client-side Web programming* tasks.

Table 5. *ChatGPT* accuracy in game programming.

ID	Topic	Blanks	Correct Blanks	Incorrect Blanks	Accuracy
1	Counter Game	11	9	2	81.82%
2	Guess Number Game	21	20	1	95.24%
3	Snake Game	17	14	3	82.35%
4	React Time Game	16	11	5	68.75%
5	Hit Mole Game	20	14	6	70%
Total (Average)		85 (17)	68 (13.6)	17 (3.4)	(79.63%)

“Guess-Number Game” achieved near perfect accuracy, at 95.2%, while “Reaction-Time Game” recorded the lowest accuracy, at 68.8%, indicating that timing-sensitive and event-driven logic become more challenging for *ChatGPT*.

10.3. Answer Patterns by AI

Through the analysis of the answers to these instances by *ChatGPT*, we identified the following five patterns, as shown in Table 6, which compares the original correct answers and the answers generated by *ChatGPT* in the five patterns.

Table 6. Comparison of answers between the original and *ChatGPT*.

Answer Pattern	Original Answer	ChatGPT Answer
Exact Match	<title>Click Game	<title>Click Game
Exact Content, Reordered	_1_: text-align: center; _2_: background-color: white;	_1_: background-color: white; _2_: text-align: center;
Format Variation	background-color: green;	background-color: #006400;
Contextual Error	if (guess > randomNumber)	if (guess < randomNumber)
Syntax Error	Enter a number between 1 and 100:</p>	Enter a number between 1 and 100:

1. **Exact match:** The output exactly matches the expected answer and is validated successfully.
2. **Exact content, reordered:** The output contains all the required information but in a different order, which is marked incorrectly under *string matching*. This issue often occurs in CSS properties that have multiple style declarations.
3. **Format variation:** Equivalent selections are expressed through different formats (e.g., using hexadecimal color codes instead of color names).
4. **Contextual error:** Incorrect answers are obtained due to incomplete contexts, such as a lack of visual reference like screenshots.
5. **Syntax error:** The output is incomplete or malformed (e.g., missing *HTML* tags), potentially resulting in rendering issues or failed validations.

10.4. Common Mistakes by AI

In addition to the five answer patterns, we identified the following three common mistakes by *ChatGPT* when responding to *HTML*, *CSS*, and *JavaScript* prompts:

1. **HTML Structure and Syntax:** *ChatGPT* sometimes generates too few *HTML* tags, which can make the structure confusing or harder to read. This happens because it was trained on a wide variety of Web content, each with different coding styles. In contrast, our algorithm intentionally blanks either the starting tag or the ending tag to help students learn and recognize the correct *HTML* structure and the tag names more clearly.
2. **Mismatched CSS Property:** In some cases, the *CSS* property returned by *ChatGPT* is syntactically correct but does not match the instructional objective in context. For example, it might generate `box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);` instead of the expected `cursor: pointer;`. The `cursor` attribute explicitly denotes interactivity, and the former is a styling attribute focused on visual enhancement. While both properties are valid, only `cursor: pointer;` meets the intended learning goal of recognizing interactive elements. In this paper, we emphasize this attribute and demonstrate it visually in the screenshots to help students make a clear connection between the code and the intended behavior.
3. **JavaScript Logic Conditions:** Sometimes, *ChatGPT* uses different logic than the original response. For example, it uses “greater than” instead of “less than”. This happens because *ChatGPT* copies what it has seen often in its training data, not necessarily what is correct for the task at hand. A source code in the proposal always follows the correct logic based on what the student is supposed to learn, ensuring consistency with the lesson’s intended objective.

These findings highlight the importance of defining robust answer validation methods when incorporating *generative AI* into educational systems.

10.5. Limitations and Future Direction of Proposal

In this section, we briefly discuss limitations of our proposal. Although the proposed algorithm systematically identifies and replaces critical phrases in a source code, it is subject to several notable constraints:

One of the primary limitations observed in our evaluation concerns the exact content, reordered types of responses generated by *ChatGPT*. While such responses are semantically correct and functionally equivalent to the intended answers, they often fail validations under our current *string-matching* mechanism due to their different order of appearance, even if the answers are grammatically correct. This issue is particularly frequent in *CSS* properties, where the order of style declarations may vary without affecting the rendered result.

As a direction for our future research, this limitation suggests the need to enhance our system with more flexible validation strategies. Since programming education does not always follow a strictly deterministic writing style, it is important to accommodate different yet valid ways of expressing the same logic. Our improvements of the proposal include implementing answer matching that supports different orders of correct answer patterns. This enables more flexible and fair validation of responses that are logically correct but syntactically varied. Such enhancements would allow the system to better handle valid variations in programming educations, where multiple correct expressions may exist for the same logical intent.

11. Conclusions

This paper proposed the *Phrase Fill-in-blank Problem (PFP)* as a novel exercise format within the *Web Programming Learning Assistant System (WPLAS)*, supporting self-study of

client-side Web programming. PFP is designed to bridge the gap between code comprehension and dynamic Web behaviors, particularly in mastering *JavaScript-based DOM manipulation* and *event-driven programming*, which are often challenging for beginners.

To facilitate scalable content creation, the automated algorithm for the generation of PFP instances was implemented using *Python*. This algorithm adopts eight general and six game-specific phrase selection rules to identify syntactically and semantically meaningful code segments from the integrated *HTML*, *CSS*, and *JavaScript* source code. This automatic generation mechanism reduces instructor workloads and supports the production of diverse, context-aware exercises.

The proposal was evaluated through two experiments involving 10 basic-topic and 5 game-based PFP instances. These were assigned to students from Okayama University, Japan, and the State Polytechnic of Malang, Indonesia. The results indicated that most students could successfully solve the instances, confirming their educational relevance and practical solvability.

Additionally, the problem-solving capability of a *generative AI* model, *ChatGPT*, was assessed on the same instances. *ChatGPT* achieved an 86.7% accuracy rate on the basic-topic exercises, demonstrating the feasibility of AI-based support. However, this also revealed the existence of multiple correct answers and stylistic variations, underscoring the importance of flexible evaluation mechanisms in automated programming assessment systems.

For future works, we aim to enhance the algorithm by incorporating semantic analysis to better capture pedagogically valuable code segments, introduce an adaptive difficulty control mechanism based on learner performance data, and improve the evaluation framework by integrating order-insensitive and equivalence-aware answer validation to accommodate diverse yet correct responses. These directions are expected to increase the adaptability of *WPLAS*, contributing to the broader goal of supporting effective self-directed learning in programming education.

Author Contributions: Conceptualization, N.F.; methodology, H.Q. and N.F.; software, H.Q.; validation, Z.L.; formal analysis, H.Q.; investigation, Z.L.; resources, Z.L.; data curation, H.Q.; writing—original draft preparation, H.Q.; writing—review and editing, H.Q.; visualization, N.F. and H.H.S.K.; supervision, N.F.; project administration, W.C.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are newly generated and are not publicly available due to educational and research use limitations.

Acknowledgments: The authors would like to thank to the students from Okayama University and State Polytechnic of Malang from answering the instances and providing us with comments. They were essential to completing this paper.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Appendix A. Source-Code Listings for Key Phrase Detection

This appendix presents the technical details and Python code fragments used in the phrase detection algorithm for PFP instance generation. These implementations illustrate how regular expressions are applied to locate key phrase elements in client-side Web programming source code.

Appendix A.6. Detection of CSS Syntax Elements

```
css_keywords_pattern = re.compile(
    r"(background-color\s*:\s*[^\s;]+;|color\s*:\s*[^\s;]+;"
    r"|border\s*:\s*[^\s;]+;|text-align\s*:\s*[^\s;]+;"
    r"|cursor\s*:\s*[^\s;]+;|align-items\s*:\s*[^\s;]+;)"
)
line = css_keywords_pattern.sub(
    lambda m: self._store_and_replace(m.group(0)), line
)
```

Figure A6. Python code for detecting CSS properties.

Appendix A.7. Detection of HTML Tags and Headings

```
title_pattern = re.compile(r"<title>.*?</title>", re.IGNORECASE)
line = title_pattern.sub(lambda m: self._store_and_replace(m.group(0)), line)

noid_pattern = re.compile(r"<(h1|h2|h3|p|div|label)>(.*?)</\1>")
line = noid_pattern.sub(_noid_replacer, line)
```

Figure A7. Python code for detecting HTML tags and heading elements.

Appendix A.8. Detection of HTML Attributes

```
# Detecting input and onclick in <button>:
input_pattern = re.compile(r"<input\s+[^\s]*>")
line = input_pattern.sub(lambda m: self._store_and_replace(m.group(0)), line)

button_pattern = re.compile(
    r'(<button[^\s]*onclick="[^\s]*"[^\s]*>)(.*?)(</button>)'
)
def _btn_replacer(m):
    original_text = m.group(2) + m.group(3)
    placeholder = self._store_and_replace(original_text)
    return m.group(1) + placeholder
line = button_pattern.sub(_btn_replacer, line)
```

Figure A8. Python code for detecting onclick and other HTML attributes.

Appendix B. Example Output Text File

This appendix provides a full example of an output text file that is automatically generated by the proposed algorithm.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0"
  >
_1_
<style>
body {
    font-family: Arial, sans-serif;
    _2_ # Specify the alignment of the text #
    _3_ # Set the background color #
}
...
.cell.active {
    _8_ # Set the background color #
}
</style>
</head>
<body>
<div id="game">
<h1> _9_
<div id="grid"></div>
<p>Score: <span id="score">0</span></p>
<button id="startButton">Start Game</button>
</div>
<script>
let score = 0;
_10_ # Declare variable to track the active cell #
_11_ # Declare variable to store the interval ID for the game loop #
_12_ # The grid container #
_13_ # The score display element #
_14_ # Make a 9 x 9 grid #
const cell = document.createElement("div");
cell.classList.add("cell");
cell.addEventListener("click", () => {
    _15_ {
        _16_
        scoreDisplay.textContent = score;
        cell.classList.remove("active");
        activeCell = null;
    }
});
grid.appendChild(cell);
}
...
</script>
</body>
</html>

```

Figure A9. Example of an output text file for a PFP instance, showing numbered blanks.

References

1. What Is JavaScript. Available online: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/What_is_JavaScript (accessed on 1 June 2025).
2. Regular Expression in Python. Available online: <https://docs.python.org/3/library/re.html> (accessed on 1 April 2025).
3. ChatGPT. Available online: <https://openai.com/chatgpt/overview/> (accessed on 1 April 2025).
4. Qi, H.; Li, Z.; Funabiki, N.; Kyaw, H.H.S.; Kao, W.C. A proposal of blank phrase selection algorithm for phrase fill-in-blank problems in web-client programming learning assistant system. In Proceedings of the 2025 1st International Conference on Consumer Technology (ICCT-Pacific), Matsue, Japan, 29–31 March 2025; pp. 1–4.

5. Smith, M.; Johnson, L.; Patel, R. Automated fill-in-the-blank exercises in online learning: Effectiveness and limitations. *Int. J. Comput. Educ.* **2017**, *25*, 134–145.
6. Jordan, M.; Ly, K.; Soosai Raj, A.G. Need a programming exercise generated in your native language? ChatGPT's got your back: Automatic generation of non-English programming exercises using OpenAI GPT-3.5. In Proceedings of the 55th ACM Technical Symposium on Computer Science Education (SIGCSE '24), Portland, OR, USA, 20–23 March 2024.
7. Denny, P.; Luxton-Reilly, A.; Prather, J.; Becker, B.A. Prompt Problems: A new programming exercise for the generative AI era. In Proceedings of the 55th ACM Technical Symposium on Computer Science Education, Portland, OR, USA, 20–23 March 2024; Volume 1, pp. 1–6.
8. Messer, M.; Paiva, J.; Hauksson, H.; Ihantola, P.; Prather, J.; Petersen, A.; Edwards, S.H.; Luxton-Reilly, A.; Becker, B.A. Automated grading and feedback tools for programming education: A systematic review. *ACM Trans. Comput. Educ.* **2024**, *24*, 1–43. [\[CrossRef\]](#)
9. Daradoumis, T.; Puig, J.M.M.; Arguedas, M.; Liñan, L.C. Analyzing students' perceptions to improve the design of an automated assessment tool in online distributed programming. *Comput. Educ.* **2019**, *128*, 159–170. [\[CrossRef\]](#)
10. Duran, R.; Rybicki, J.M.; Sorva, J.; Hellas, A. Exploring the value of student self-evaluation in introductory programming. In Proceedings of the 2019 ACM Conference on International Computing Education Research, Toronto, ON, Canada, 12–14 August 2019; pp. 121–130.
11. Papadakis, S. Evaluating a game-development approach to teach introductory programming concepts in secondary education. *Int. J. Technol. Enhanc. Learn.* **2020**, *12*, 127–145. [\[CrossRef\]](#)
12. Gulec, U.; Yilmaz, M.; Yalcin, A.D.; O'Connor, R.V.; Clarke, P.M. Cengo: A web-based serious game to increase the programming knowledge levels of computer engineering students. In Proceedings of the EuroSPI 2019, Edinburgh, UK, 18–20 September 2019; Springer: Cham, Switzerland, 2019; pp. 237–248.
13. Estévez-Ayres, I.; Callejo, P.; Hombrados-Herrera, M.Á.; Alario-Hoyos, C.; Kloos, C.D. Evaluation of LLM tools for feedback generation in a course on concurrent programming. *Int. J. Artif. Intell. Educ.* **2024**, *35*, 774–790. [\[CrossRef\]](#)
14. Ma, B.; Chen, L.; Konomi, S. Enhancing programming education with ChatGPT: A case study on student perceptions and interactions in a Python course. In *International Conference on Artificial Intelligence in Education*; Springer: Cham, Switzerland, 2024.
15. Doughty, J.; Prather, J.; Teague, D.; Zingaro, D.; Petersen, A.; Becker, B.A. A comparative study of AI-generated (GPT-4) and human-crafted MCQs in programming education. In Proceedings of the 26th Australasian Computing Education Conference, Sydney, Australia, 29 January–2 February 2024; pp. 1–8.
16. Ghimire, A.; Edwards, J. Coding with AI: How are tools like ChatGPT being used by students in foundational programming courses. In Proceedings of the International Conference on Artificial Intelligence in Education, Recife, Brazil, 8–12 July 2024; pp. 259–267.
17. Silva, C.A.G.D.; Ramos, F.N.; De Moraes, R.V.; Santos, E.L.D. ChatGPT: Challenges and benefits in software programming for higher education. *Sustainability* **2024**, *16*, 1245. [\[CrossRef\]](#)
18. Popovici, M.D. ChatGPT in the classroom: Exploring its potential and limitations in a functional programming course. *Int. J. Hum.-Comput. Interact.* **2024**, *40*, 7743–7754. [\[CrossRef\]](#)
19. Rahman, M.M.; Watanobe, Y. ChatGPT for education and research: Opportunities, threats, and strategies. *Appl. Sci.* **2023**, *13*, 5783. [\[CrossRef\]](#)
20. Lo, C.K. What is the impact of ChatGPT on education? A rapid review of the literature. *Educ. Sci.* **2023**, *13*, 410. [\[CrossRef\]](#)
21. Strielkowski, W.; Grebennikova, V.; Lisovskiy, A.; Rakhimova, G.; Vasileva, T. AI-driven adaptive learning for sustainable educational transformation. *Sustain. Dev.* **2025**, *33*, 1921–1947. [\[CrossRef\]](#)
22. Chang, C.I.; Choi, W.C.; Choi, I.C. A systematic literature review of the opportunities and advantages for AIGC (OpenAI ChatGPT, Copilot, Codex) in programming course. In Proceedings of the 7th International Conference on Big Data and Education (ICBDE '24), Fukuoka, Japan, 24–26 September 2024.
23. Qi, H.; Li, Z.; Funabiki, N.; Kyaw, H.H.S.; Kao, W.C. A blank element selection algorithm for element fill-in-blank problems in client-side web programming. *Eng. Lett.* **2024**, *32*, 684–700.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.